# A ROBUST FRAMEWORK FOR DEEPFAKE DETECTION USING ADVANCED NEURAL ARCHITECTURES AND GENERALIZATION TECHNIQUE

submitted in partial fulfilment of the requirements of the degree of

**BACHELOR OF COMPUTER APPLICATIONS**

**to the**

**Thiruvalluvar University, Serkkadu, Vellore – 632 115.**

**by**


| | |
|---|---|
| **ROSHITHA L.G.** | **(Reg.no: 34624U09077)** |
| **SABITHA MARIYAM A.R.** | **(Reg.no: 34624U09080)** |
| **SANJANA SRI M.** | **(Reg.no: 34624U09082)** |
| **SARIGA SREE B.** | **(Reg.no: 34624U09083)** |
| **SHAFIYA AYMAN K.S.** | **(Reg.no: 34624U09085)** |


Under the supervision of

**Ms. SHARMI S., M.C.A.,**

**DEPARTMENT OF COMPUTER APPLICATIONS**


**M.M.E.S. WOMEN'S ARTS & SCIENCE COLLEGE**

**(AFFILIATED TO THIRUVALLUVAR UNIVERSITY)**

**Hakeem Nagar, Melvisharam –632 509**


**OCTOBER-2026**

# DECLARATION

I hereby certify that the project entitled "**A ROBUST FRAMEWORK FOR DEEPFAKE DETECTION USING ADVANCED NEURAL ARCHITECTURES AND GENERALIZATION TECHNIQUE**" submitted by **ROSHITHA L.G. (34624U09077), SABITHA MARIYAM A.R. (34624U09080), SANJANA SRI M. (34624U09082), SARIGA SREE B. (34624U09083), SHAFIYA AYMAN K.S. (34624U09085**), in partial fulfilment of the requirement for the award of the degree of **B.C.A** submitted at **M.M.E.S Women's Arts and Science College** is an authentic record of our own work carried out during the period from June 2026 to October 2026 under the guidance of **Ms. S. SHARMI, M.C.A., Department of Computer Applications.** The matter presented in this project has not formed the basis for the award of any other degree, diploma, fellowship or any other similar titles.

**SIGNATURE OF THE STUDENTS**

1.

2.

3.

4.

5.

**PLACE:**

**DATE:**

# BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**A ROBUST FRAMEWORK FOR DEEPFAKE DETECTION USING ADVANCED NEURAL ARCHITECTURES AND GENERALIZATION TECHNIQUE**" is the bonafide work carried out by **ROSHITHA L.G. (34624U09077), SABITHA MARIYAM A.R. (34624U09080), SANJANA SRI M. (34624U09082), SARIGA SREE B. (34624U09083), SHAFIYA AYMAN K.S. (34624U09085)**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications submitted at **M.M.E.S WOMEN'S ARTS AND SCIENCE COLLEGE** is an authentic record of my own work, carried out during a June 2026 to October 2026 under the guidance of **Ms. SHARMI S, M.C.A, Department of Computer Applications.** The major project viva-voice examination has been held on _____

**SIGNATURE OF THE GUIDE**

**SIGNATURE OF**

**HEAD OF DEPARTMENT**

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

# ACKNOWLEDGEMENT

# ACKNOWLEDGEMENT

First and foremost, I bow in gratitude to the Almighty for His endless blessings and mercy, which enabled us to carry out and complete this project successfully.

I am grateful to our respected General Secretary **Alijanab Alhaj. Dr.S.Ziauddeen Ahmed Sahib** and Secretary / Correspondent **Aliianab Alhaj Dr.K.Anees Ahmed Sahib, B.A., D.Lit.,** and to our esteemed Principal **Dr. Freda Gnanaselvam, M.A., M.B.A., M.M.M., Ph.D.,** for providing me with all facilities and a supportive environment to accomplish this work.

I am thankful to our Head of the Department, **Mrs. N. Nithyalakshmi, M.Sc., M.Phil., SET., (Ph.D.).** for giving the valuable guidance and advice, patiently for time to without which the project could not have been success.

I would like to take this opportunity to express our heartfelt and sincere thanks to my project guide **Ms. S. SHARMI, M.C.A.,** for her continuous guidance, constructive feedback. and motivation.

Finally, we wish to express our deep sense of gratitude to our beloved parents, our brothers and sisters, and my close friends for their constant encouragement, support, and prayers that gave me the strength to successfully complete this endeavour.

# ABSTRACT

# ABSTRACT

The rapid growth of deepfake technology has created significant challenges related to privacy, security, misinformation, identity theft, and digital trust. Manipulated images can be used to spread false information, damage reputations, and support fraudulent activities. Therefore, there is a growing need for an intelligent and reliable system that can automatically identify and classify deepfake images.

This project proposes a Deepfake Image Detection System using a hybrid deep learning approach that combines ResNet-50 and LSTM. ResNet-50 is used to extract important facial and visual features from images, while LSTM helps identify hidden patterns and inconsistencies in the extracted features. The combination of these models enables the system to distinguish between real and manipulated images more effectively.

The proposed system includes data collection, image preprocessing, facial feature extraction, deepfake classification, secure data handling, and real-time notification alerts. These components are designed to improve detection accuracy, reduce false classifications, and provide a faster and more reliable verification process. The system is also developed with a user-friendly approach so that it can be adapted for applications such as social media verification, news authentication, digital forensics, and security monitoring.

Unlike traditional detection methods that depend mainly on manual inspection or basic image processing techniques, the proposed deep learning-based approach can automatically learn complex visual features and detect subtle manipulations that may not be easily identified by humans. The hybrid architecture aims to improve the reliability and efficiency of deepfake detection while supporting secure handling of digital content.

Overall, this project provides a practical technological solution for identifying manipulated images and reducing the potential misuse of deepfake technology. It contributes towards creating a safer and more trustworthy digital environment by supporting the verification and authentication of digital images.

# TABLE OF CONTENT


| CHAPTER NO. | TITLE | PAGE NO |
|---|---|---|
| | **ABSTRACT** | |
| | **ACKNOWLEDGEMENT** | |
| | **LIST OF FIGURES** | |
| | **LIST OF ABBREVIATION** | |
| **1** | **INTRODUCTION** | |
| | 1.1 Overview of the Project | |
| | 1.2 Problem Definition | |
| | 1.3 Scope of the Project | |
| **2** | **SYSTEM ANALYSIS** | |
| | 2.1 Existing System | |
| | 2.2 Proposed System | |
| | 2.3 Comparison of Existing and Proposed Systems | |
| | 2.4 Feasibility Study | |
| **3** | **DEVELOPMENT ENVIRONMENT** | |
| | 3.1 Functional Requirements | |
| | 3.2 Hardware Requirement | |
| | 3.3 Software Requirements | |
| **4** | **SYSTEM DESIGN** | |
| | 4.1 Architecture Diagram | |
| | 4.2 Data Flow Diagram | |

| | | |
|---|---|---|
| | 4.3 Use Case Diagram | |
| | 4.4 Sequence Diagram | |
| | 4.5 Class Diagram | |
| | 4.6 Activity Diagram | |
| | 4.7 State Flow Diagram | |
| **5** | **IMPLEMENTATION** | |
| | 5.1 System Implementation | |
| | 5.2 Modules Description | |
| **6** | **SYSTEM TESTING** | |
| | 6.1 Testing strategy | |
| | 6.2 Test Environment | |
| | 6.3 Functional (Black-Box) Test Cases | |
| | 6.4 Test Result Summary | |
| **7** | **RESULTS & DISCUSSIONS** | |
| | 7.1 Functional Review | |
| | 7.2 Security Analysis | |
| | 7.3 Performance Analysis | |
| | 7.4 Usability Observations | |
| | 7.5 Sample Output | |
| **8** | **LIMITATIONS AND RISK ANALYSIS** | |
| | 8.1 Password Strength Dependency | |
| | 8.2 Local Share Storage Risk | |
| | 8.3 No Built-in Key Store | |
| | 8.4 Denial of Service via Share Deletion | |
| **9** | **LITERATURE SURVEY** | |
| | 9.1 Deepfake Image Detection Using Recurrent Neural Networks | |

| | | |
|---|---|---|
| | 9.2 Exposing Deep Fakes Using Biological Signals | |
| | 9.3 Deep Fake Detection with Spatiotemporal Features | |
| | 9.4 FakeForensics++: Learning to Detect Manipulated Facial Images | |
| | 9.5 Deepfake Detection: A Survey | |
| **10** | **APPENDECIES** | |
| | APPENDIX A: Source Coding's | |
| | 10.1 Technology Stack | |
| | APPENDIX B: Screenshots | |
| | 10.2 Error Handling Strategy | |
| | APPENDIX C: Glossary of Terms | |
| **11** | **CONCLUSION AND FUTURE WORK** | |
| | 11.1 Conclusion | |
| | 11.2 Learning Outcomes | |
| | 11.3 Future Enhancement | |
| **12** | **REFERENCES** | |

# LIST OF FIGURES


| S.NO | TITLE | PAGE NO |
|---|---|---|
| 1 | Architecture Diagram | |
| 2 | Data Flow Diagram (DFD) | |
| 3 | Use Case Diagram | |
| 4 | Sequence Diagram | |
| 5 | Class Diagram | |
| 6 | Activity Diagram | |
| 7 | State Flow Diagram | |
| 8 | Module 1 – Data Collection | |
| 9 | Module 2 – Image Preprocessing | |
| 10 | Module 3 – Feature Extraction | |
| 11 | Module 4 – Pattern Analysis | |
| 12 | Module 5 – deepfake Detection | |
| 13 | Module 6 – Database Management | |
| 14 | Module 7 – Result & Notification | |
| 15 | Module 8 - Security | |
| 16 | Screenshot - 01 | |
| 17 | Screenshot – 02 | |
| 18 | Screenshot – 03 | |
| 19 | Screenshot – 04 | |
| 20 | Screenshot - 05 | |

# LIST OF ABBREVIATIONS

- CRUD – Create, Read, Update, Delete
- DB - Database
- IoT - Internet of Things
- JWT - JSON Web Token
- ML - Machine Learning
- AI - Artificial Intelligence
- ODM – Object Data Modelling
- RESTful API / REST – Representational State Transfer
- UI - User Interface
- UX - User Experience
- NLP - Natural Language Processing
- SMB - Small and Medium-sized Business
- CDN - Content Delivery Network
- CORS - Cross-Origin Resource Sharing
- VS Code - Visual Studio Code
- JS - JavaScript
- Node.js - JavaScript runtime
- React.js - JavaScript library for UI
- Express.js - Web framework for Node.js
- MongoDB – NoSQL database

# INTRODUCTION

# CHAPTER 1
# INTRODUCTION

The rapid growth of deepfake technology has created serious challenges such as misinformation, privacy violations, identity theft, fraud, cybercrime, and loss of digital trust. Deepfake images can be created using advanced Artificial Intelligence techniques and may appear realistic to users. Therefore, an intelligent system is needed to automatically identify manipulated images and verify their authenticity.

This project proposes a Deepfake Image Detection System using a Hybrid Deep Learning Approach. The system combines ResNet-50 for extracting important visual and facial features and LSTM for identifying hidden patterns and inconsistencies. By combining these techniques, the system aims to improve the reliability of deepfake image detection.

The proposed system includes data collection, preprocessing, face detection, feature extraction, classification, secure encryption, and real-time fake image notification alerts. Image preprocessing helps improve the quality of input data, while the hybrid model analyses important features to classify an image as real or fake.

The system is designed to provide a fast, accurate, secure, and user-friendly solution for verifying image authenticity. It can be useful in areas such as social media verification, news authentication, digital forensics, law enforcement, and online security.

Unlike traditional manual inspection and basic image-processing methods, the proposed deep learning approach can automatically learn complex features and detect subtle manipulations. It also aims to reduce false classifications and improve the overall efficiency of detection.

The system can be further enhanced in the future to support deepfake video and audio detection, mobile applications, browser extensions, and large-scale real-time monitoring. Overall, this project provides a practical approach to detecting manipulated digital content and supports a safer and more trustworthy digital environment.

## 1.1 OVERVIEW OF THE PROJECT

Deepfake technology uses Artificial Intelligence (AI) and Deep Learning to create realistic fake images, videos, and audio. Although it has useful applications in entertainment and education, its misuse can cause privacy issues, misinformation, identity theft, fraud, and reputational damage.

This project, "Deepfake Image Detection System using Hybrid Deep Learning Approach," aims to detect whether an image is real or fake. The proposed system combines ResNet-50 for feature extraction and LSTM for pattern analysis to provide accurate and fast deepfake detection.

### 1.1.1 OBJECTIVES OF THE PROJECT

- To develop a hybrid deep learning model using ResNet-50 and LSTM.
- To detect fake and manipulated images with high accuracy.
- To focus specifically on still image deepfake detection.
- To use preprocessing techniques such as face detection, cropping, normalization, and augmentation.
- To provide fast and user-friendly image detection.
- To provide an alert when a fake image is detected.

## 1.2 PROBLEM STATEMENT

Existing deepfake detection methods often focus mainly on videos and may not detect modern AI-generated fake images accurately. Manual inspection and simple detection techniques are also unreliable. Therefore, there is a need for a fast, accurate, secure, and user-friendly system that can detect deepfake still images and reduce the spread of misinformation and digital fraud.

## 1.3 SCOPE OF THE PROJECT

The proposed system can be useful in:

- **Social Media:** Detecting and flagging manipulated images.
- **Digital Forensics**: Supporting verification of digital evidence.
- **Journalism**: Checking images before publishing news.
- **Banking and KYC**: Detecting fake images used for identity fraud.
- **Corporate Security**: Protecting organizations from manipulated images.
- **Future Scope:** The system can be extended to detect deepfake videos and audio and can be developed as a mobile application or browser extension.

# SYSTEM ANALYSIS

# CHAPTER 2
# SYSTEM ANALYSIS

## 2.1 EXISTING SYSTEM

Existing deepfake detection systems use Machine Learning, CNNs, and Deep Learning to classify media as real or fake. Common models include VGG-16, InceptionV3, and XceptionNet, trained on datasets such as FaceForensics++ and UADFV.

Many systems focus mainly on video analysis, detecting visual artifacts, frame inconsistencies, abnormal blinking, lighting differences, and lip-sync problems. However, these methods often struggle with new and unseen deepfake techniques and real-world images.

### 2.1.1 DISADVANTAGES OF EXISTING SYSTEM

1. **Limited Still-Image Detection**: Most systems focus on videos and cannot effectively analyze individual images.
2. **Lower Accuracy**: Models may produce false positives and false negatives when tested on new datasets.
3. **Slow Processing:** Complex models often require high computing power and are not suitable for real-time detection.
4. **Limited Feature Extraction:** Traditional methods may fail to detect subtle pixel-level manipulations.
5. **Dataset Dependency**: Performance depends heavily on the size, quality, and diversity of training datasets.
6. **Lack of Security and Alerts**: Many systems provide only a Real/Fake result without encryption, authentication, or notification features.
7. **Adversarial Vulnerability:** Some models can be affected by small changes or noise designed to fool the detector.

Therefore, a hybrid, secure, accurate, and real-time deepfake detection system is needed, especially for still images.

## 2.2 PROPOSED SYSTEM

The proposed system is designed to overcome the limitations of existing deepfake detection methods by using a Hybrid Deep Learning Framework. It combines ResNet-50 for extracting important visual features and LSTM for analyzing hidden patterns and inconsistencies. The system mainly focuses on detecting deepfake still images.

The system collects images from datasets such as FaceForensics++, Celeb-DF, and DFDC. The images are preprocessed using face detection, cropping, resizing, normalization, and augmentation. ResNet-50 extracts facial features, textures, edges, and other visual patterns. These features are then analyzed by LSTM to classify the image as Real or Fake.

The system also provides real-time detection, secure encryption, and fake image notification alerts. Performance can be evaluated using Accuracy, Precision, Recall, F1-Score, and Confusion Matrix.

### 2.2.1 Key Features

The system combines ResNet-50 and LSTM to extract features and analyze patterns for deepfake detection.

It provides real-time processing, still-image detection, improved accuracy, comprehensive evaluation, and fake-image alerts.

**1**. **Hybrid Architecture:** Combines ResNet-50 and LSTM for better feature and pattern analysis.

**2**. **Still Image Detection**: Focuses specifically on identifying manipulated images.

**3. Real-Time Detection**: Provides quick results for uploaded images.

**4**. **Data Preprocessing**: Uses face detection, resizing, normalization, and augmentation.

**5**. **Security and Alerts**: Supports secure data handling and fake-image notifications.

**6**. **Performance Evaluation**: Uses standard metrics to measure detection performance.

### 2.2.2 Advantages of Proposed System

The proposed system provides an efficient and automated approach for detecting deepfake images using a hybrid deep learning model. It improves the reliability of detection by combining feature extraction, pattern analysis, and real-time classification.

- Better feature extraction using ResNet-50.
- Improved pattern analysis using LSTM.
- Transfer learning reduces training time and data requirements.
- Data augmentation helps reduce overfitting.
- Can be scaled for different datasets and applications.
- Provides a simple and user-friendly interface.
- Suitable for applications such as social media, journalism, and digital forensics.

## 2.2.3 DISADVANTAGES OF PROPOSED SYSTEM

The proposed system may require large datasets and considerable computational resources for training and processing.

.

**1**. **High Computational Requirement:** ResNet-50 and LSTM may require considerable processing power during training.

**2**. **Dataset Dependency**: Detection performance depends on the quality and diversity of training data.

**3**. **New Deepfakes:** The system may require retraining to detect newly developed deepfake techniques.

**4**. **False Detection:** Highly realistic or low-quality images may sometimes be incorrectly classified.

**5**. **Security Overhead**: Encryption and authentication can add additional processing time.

**6**. **Still Image Limitation**: The current system mainly focuses on images and does not directly detect deepfake videos or audio.

## 2.3 COMPARISON OF EXISTING AND PROPOSED SYSTEMS

| S.NO | Parameter | Existing System | Proposed System |
|---|---|---|---|
| 1 | **Detection** | Mostly video | Still images |
| 2 | **Model** | CNN / Single Model | ResNet-50 + LSTM |
| 3 | **Features** | Basic extraction | Deep feature extraction |
| 4 | **Accuracy** | More errors | Improved accuracy |
| 5 | **Processing** | Often slow | Faster detection |
| 6 | **Preprocessing** | Limited | Advanced preprocessing |
| 7 | **Security** | Limited | Encryption & authentication |
| 8 | **Alerts** | Usually unavailable | Fake-image alerts |
| 9 | **Evaluation** | Basic metrics | Multiple performance metrics |
| 10 | **Future Scope** | Limited | Video, audio, mobile & web |

Table 2.1: Comparison of Existing System and Proposed System

## 2.4 FEASIBILITY STUDY

The feasibility study evaluates whether the proposed Deepfake Image Detection System using ResNet-50 and LSTM is technically, economically, operationally, legally, and practically possible.

### 2.4.1 TECHNICAL FEASIBILITY

The system can be developed using open-source technologies such as Python, React.js, Flask/Django, TensorFlow, Keras, and OpenCV. Pre-trained ResNet-50 and free cloud platforms like Google Colab and Kaggle reduce hardware and training requirements. Required datasets are also available for research purposes.

### 2.4.2 OPERATIONAL FEASIBILITY

The system provides a simple interface where users can upload an image and receive a Real/Fake result with a confidence score. Alerts and an admin dashboard can also support monitoring and verification.

### 2.4.3 ECONOMIC FEASIBILITY

The project is low-cost because most software, libraries, datasets, and cloud resources are freely available. Only optional deployment or hosting may require additional cost.

### 2.4.4 LEGAL AND ETHICAL FEASIBILITY

The system is designed to detect deepfakes rather than create them. User data will be protected using secure methods, and datasets will be used according to their research and licensing requirements.

### 2.4.5 SCHEDULE FEASIBILITY

The project can be completed within 6 months, covering literature survey, data collection, preprocessing, model development, web integration, testing, and documentation.

# SYSTEM REQUIREMENTS

# CHAPTER 3
# SYSTEM REQUIREMENTS

## 3.1 FUNCTIONAL REQUIREMENTS

| S.No. | Functional Requirement | Description |
|---|---|---|
| 1 | Graphical User Interface | The system shall provide a GUI for users to interact with the deepfake detection system. |
| 2 | Image Upload | The system shall allow users to upload an image for deepfake analysis. |
| 3 | Image Preprocessing | The system shall resize, normalize, and prepare the uploaded image for analysis. |
| 4 | Feature Extraction | The system shall use ResNet-50 to extract important spatial and facial features. |
| 5 | Deepfake Detection | The system shall analyze the extracted features using the proposed ResNet-50 and LSTM hybrid model. |
| 6 | Image Classification | The system shall classify the uploaded image as Real or Fake. |
| 7 | Result Display | The system shall display the detection result to the user. |
| 8 | Notification Alert | The system shall provide an alert when a fake image is detected. |
| 9 | Data Security | The system shall protect sensitive data using encryption techniques. |
| 10 | Model Evaluation | The system shall evaluate the model using diverse real and manipulated datasets. |

Table 3.1: Functional Requirements

## 3.2 HARDWARE REQUIREMENTS

.

| S.NO | HARDWARE COMPONENT | SPECIFICATION |
|---|---|---|
| 1 | Processor | Pentium IV 2.4 GHz or higher |
| 2 | RAM | 512 MB |
| 3 | Hard disk | 40GB |
| 4 | Monitor | 14-inch colour monitor |
| 5 | Input device | Optical mouse |

Table 3.2: Hardware Requirements

### 3.2.1 Processor

Pentium IV 2.4 GHz or higher processor is used for executing the application and performing processing operations.

### 3.2.2 RAM

A minimum of 512 MB RAM is specified in the main hardware-requirement section to support application execution.

### 3.2.3 Hard Disk

A 40 GB hard disk is specified for storing the application, datasets, and related files.

### 3.2.4 Monitor

A 14-inch colour monitor is specified for displaying the application's interface, inputs, outputs, and detection results.

### 3.2.5 Input Device

An optical mouse is specified as the input device for interacting with the application's graphical user interface.

## 3.3 SOFTWARE REQUIREMENTS

| S.NO | Category | Technology |
|---|---|---|
| 1 | Programming Language | Python |
| 2 | Front-End | HTML, CSS |
| 3 | Back-End Framework | Django |
| 4 | Database | MySQL |
| 5 | Database Connector | MySQLclient |
| 6 | Local Server Environment | WampServer |
| 7 | Deep Learning | CNN, ResNet-50, LSTM |
| 8 | Deep Learning Framework | Keras / TensorFlow |
| 9 | Model Architecture | ResNet-50 + LSTM |
| 10 | Data Processing | Image resizing, normalization, augmentation |
| 11 | Development/Testing | Web Browser, particularly Chrome |
| 12 | Operating System mentioned in report | Windows |
| 13 | Security | Encryption techniques |
| 14 | Output | Real/Fake classification and notification alert |

Table 3.3: Software Requirements

### 3.3.1 Front-End Development

#### 3.3.1.1 HTML

HTML (Hypertext Markup Language) is used to create the structure of the web interface.

In the proposed system, HTML can be used to create:

- Home page
- User interface
- Image-upload form
- File-selection controls
- Submit button
- Detection-result page
- Real/Fake result display
- Notification area
- Navigation elements

The HTML layer acts as the presentation structure through which the user interacts with the deepfake detection application.

#### 3.3.1.2 CSS

CSS (Cascading Style Sheets) is used to control the appearance and layout of the web application.

CSS can be used for:

- Page layout
- Buttons
- Upload box
- Forms
- Result cards
- Text formatting
- Backgrounds
- Images
- Responsive layouts
- Alert messages
- User-friendly interface design

#### 3.3.1.3 Front-End Functionality

The frontend is responsible for communicating with the user.

The main frontend functions are:

1. Display the application interface.
2. Allow the user to upload an image.
3. Provide a button to start detection.
4. Send the uploaded file to the backend.
5. Display processing status.
6. Display the final prediction.
7. Display notification alerts when a fake image is detected.

## 3.3.2 Back-End Development

#### 3.3.2.1 Python

Python is the primary programming language used for development.

Python is suitable for this project because it supports:

- Web development
- Machine learning
- Deep learning
- Image processing
- Database connectivity
- Data preprocessing
- Model implementation
- Automation

The project report identifies Python as the coding language.

Python acts as the connecting programming language between the web application and the deep-learning model.

## 3.3.3 Django Framework

Django is the backend web framework used for developing the web application. Django is a Python-based web framework designed for rapid development of database-driven applications. The report specifically lists Django as one of the software requirements.

Main responsibilities of Django:

- URL routing
- HTTP requests
- Image upload
- Form processing
- Backend logic
- Database communication
- User interaction
- Result generation
- Template rendering
- Application security

Django follows the **MVT architecture**:

**M – Model**

The Model represents the application's data and database structure.

**V – View**

The View contains the application logic.

It receives the uploaded image, processes the request and obtains the prediction.

**T – Template**

The Template represents the frontend page displayed to the user.

It uses HTML and CSS to present:

- Upload interface
- Results
- Alerts
- Detection information

## 3.3.4 Database Management System

### 3.3.4.1 MySQL

MySQL is the relational database management system specified in the project report.

The database can be used to maintain application-related information such as:

- User information
- Uploaded image information
- Detection results
- Prediction status
- Detection date/time
- Processing information

A possible database structure is:

| Field | Description |
|---|---|
| Detection_ID | Unique detection identifier |
| Image_Name | Uploaded image name |
| Prediction | Real/Fake result |
| Confidence | Model confidence value |
| Date_Time | Detection date and time |

Table 3.4: MySQL Data Structure

#### 3.3.4.2 MySQL client

MySQL client provides connectivity between the Python/Django application and MySQL database.

Its role is to allow the backend application to communicate with MySQL.

## 3.3.5 WampServer

WampServer is included in the project's software requirements.

It provides a local development environment for web application development and database services. The report specifically lists WampServer 2.4.

Its purpose in the development environment is to support local server/database operations while developing and testing the application.

## 3.3.6 Development Tools

#### 3.3.6.1Code Editor / IDE

A code editor or IDE is required for writing and maintaining:

- Python code
- Django files
- HTML files
- CSS files
- Model implementation
- Database-related code

### 3.3.7 Web Browser

A web browser is required to access and test the application.

The project report identifies **Chrome** as the particularly preferred browser for testing.

The browser is responsible for:

- Loading the web interface
- Uploading images
- Sending requests
- Displaying results
- Displaying notification messages

### 3.3.8 Deep Learning Environment

The main intelligence of the proposed system is provided by deep learning.

The project uses a hybrid architecture combining:

#### 3.3.8.1 ResNet-50

ResNet-50 (Residual Network-50) is the primary feature-extraction model used in the proposed system.

The project report explains that ResNet was developed to address the difficulty of training very deep neural networks. It uses residual blocks and skip connections to improve information and gradient flow through the network.

The report also discusses ResNet-50 through Keras and lists the ResNet family including ResNet50, ResNet50V2, ResNet101, ResNet101V2, ResNet152 and ResNet152V2.

#### 3.3.8.2 LSTM

LSTM (Long Short-Term Memory) is a type of Recurrent Neural Network (RNN).

It is designed to learn dependencies within sequential data.

The report describes LSTM as a model capable of learning order dependencies and maintaining information over longer intervals.

**LSTM Structure**

An LSTM mainly contains three important gates:

**1. Forget Gate**

Determines what information should be removed from the previous memory.

**2. Input Gate**

Determines which new information should be added to memory**.**

**3. Output Gate**

Determines what information should be produced as output.

### 3.3.9 Convolutional Neural Network (CNN)

A **Convolutional Neural Network (CNN)** is a deep-learning architecture widely used for image processing and computer vision.

CNNs automatically learn visual characteristics from images.

For deepfake detection, CNN-based architectures can learn visual characteristics associated with authentic and manipulated images.

The project's abstract describes the use of CNN and deep-learning techniques to extract subtle facial and visual characteristics.

### 3.3.10 Keras

Keras is the deep-learning library/framework referred to in the report for working with ResNet architectures.

The report specifically states that Keras is an open-source deep-learning library capable of running on top of TensorFlow and discusses using ResNet with Keras.

Keras simplifies the development of neural networks by providing ready-to-use components such as:

- Neural network layers
- Activation functions
- Optimizers
- Loss functions
- Pre-trained models
- Training functions

### 3.3.11 TensorFlow

TensorFlow provides the computational framework used by Keras for deep-learning operations.

# SYSTEM DESIGN

# CHAPTER 4

# SYSTEM DESIGN

System design translates the requirements identified in the previous chapters into a concrete architecture. This chapter presents the system architecture (protection and recovery flows), data-flow diagrams, UML use-case, sequence, class, activity, and state-flow diagrams, followed by a detailed description of each of the four system modules.

## 4.1 ARCHITECTURE DIAGRAM

The proposed system uses a hybrid deep learning architecture combining ResNet-50 and LSTM for deepfake image detection.

The system starts with an input image uploaded by the user. The input image is validated and pre-processed before analysis.

Preprocessing includes:

- Image resizing
- Normalization
- Frame extraction
- Data augmentation

The pre-processed image is given to ResNet-50 for feature extraction. The extracted features are passed to LSTM, which analyzes important patterns and relationships. The hybrid model classifies the image as Real or Fake (Deepfake). If it is fake, a notification alert is generated. This architecture enables efficient and reliable deepfake detection.

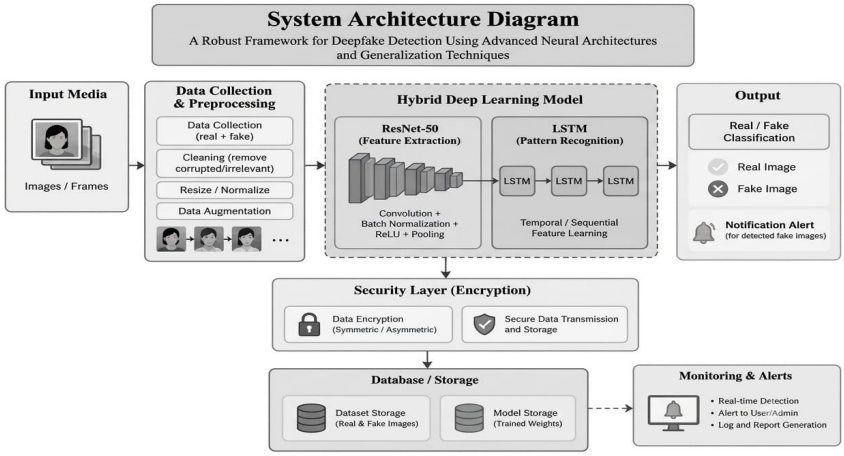



Figure 4.1

## 4.2 DATA FLOW DIAGRAM (DFD):

The process starts when the user uploads an image to the system. The system validates the uploaded image to ensure it can be processed. The validated image is passed to the preprocessing stage.

Preprocessing includes:

- Image resizing
- Normalization
- Frame extraction
- Data augmentation

The pre-processed image is given to the ResNet-50 model. ResNet-50 extracts important visual features from the image. The extracted features are transferred to the LSTM model. LSTM performs pattern analysis on the extracted features. The hybrid ResNet-50 + LSTM model performs deepfake classification. The system generates the result as Real or Fake/Deepfake. If the image is classified as Fake, a notification alert is provided to the user.

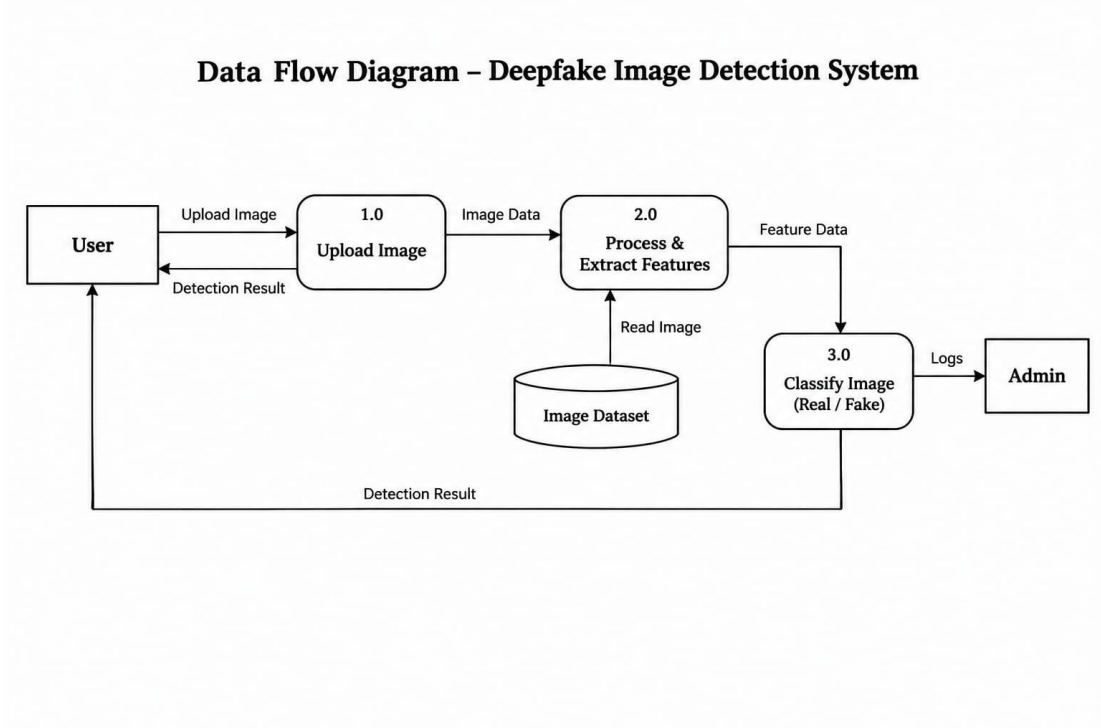



Figure 4.2

## 4.3 USE CASE DIAGRAM

The main actor of the system is the User. The user uploads an image for deepfake detection. The system validates the uploaded image.

The system performs preprocessing, including:

- Image resizing
- Normalization
- Frame extraction
- Data augmentation

The ResNet-50 model extracts important features from the image. The extracted features are analysed using the LSTM model. The hybrid ResNet-50 + LSTM model identifies possible image manipulation. The system performs deepfake classification.

The classification result is displayed to the user as:

- Real Image
- Fake/Deepfake Image
- If the image is Real, the system displays the corresponding result.
- If the image is Fake, the system generates a notification alert to inform the user.

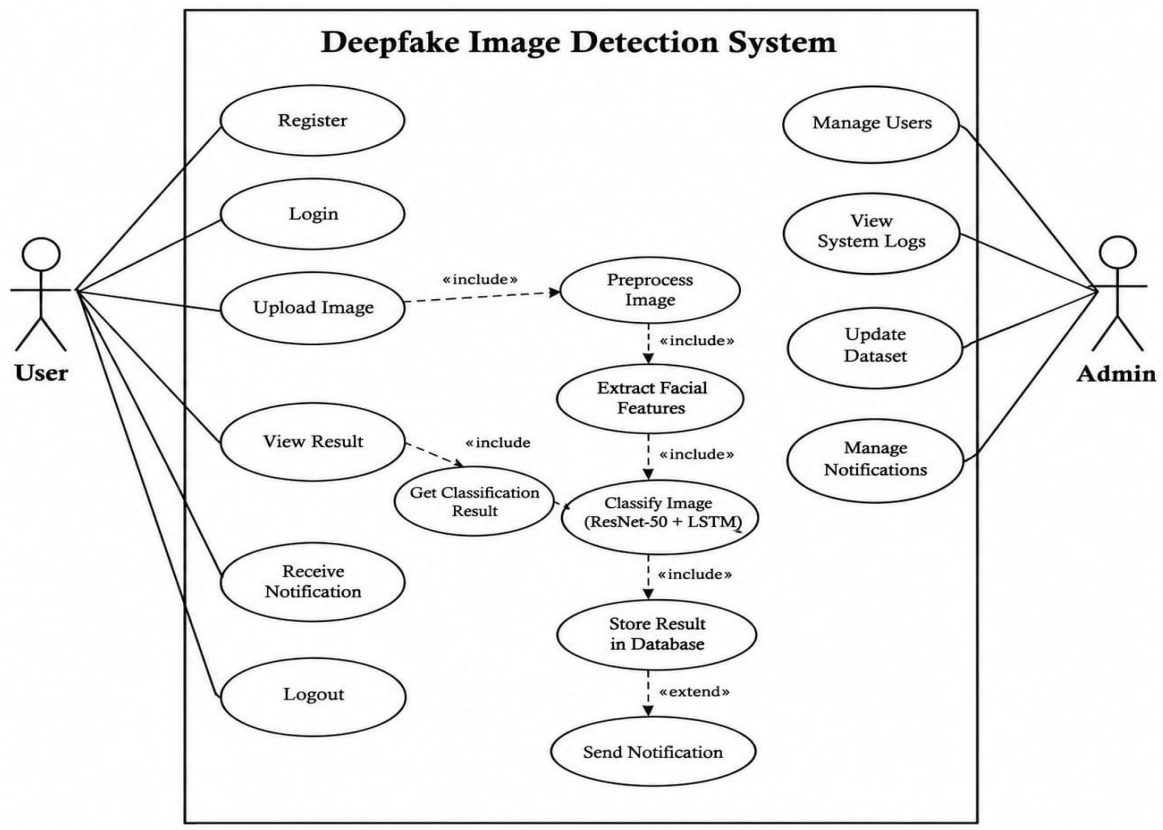



Figure 4.3

## 4.4 SEQUENCE DIAGRAM

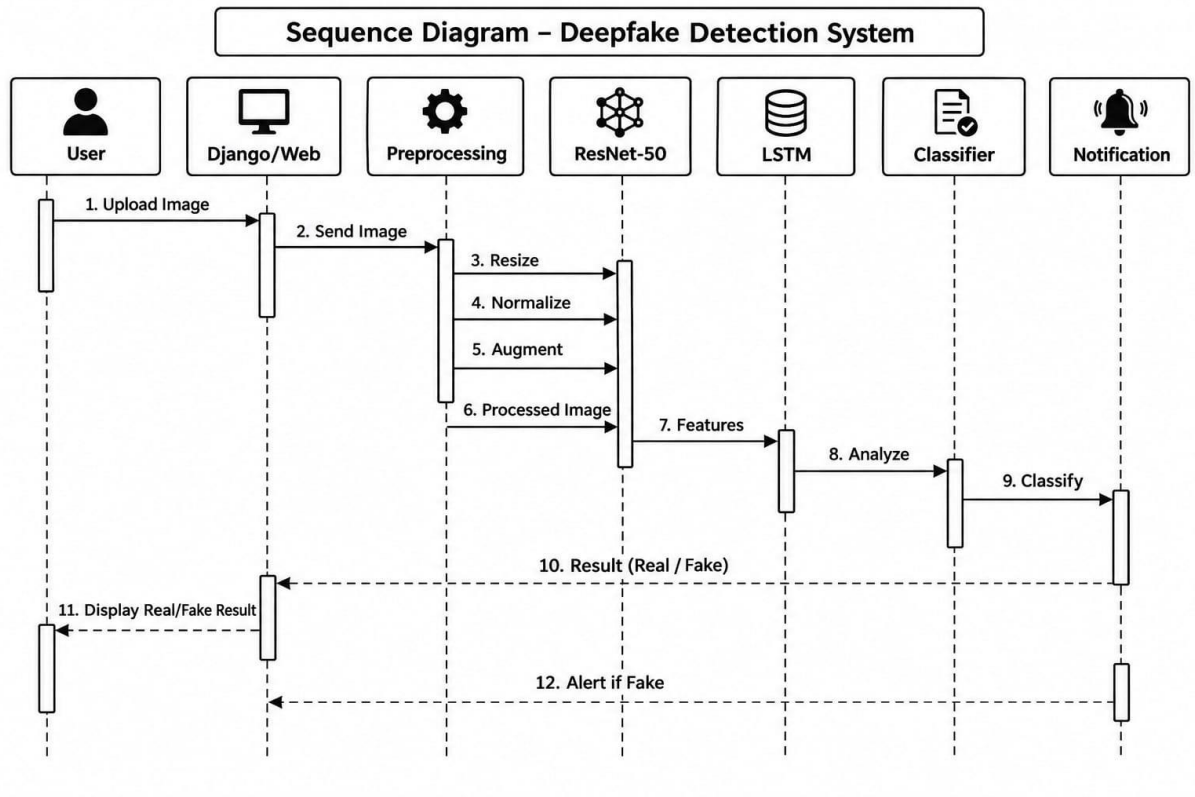



Figure 4.4

## 4.5 CLASS DIAGRAM

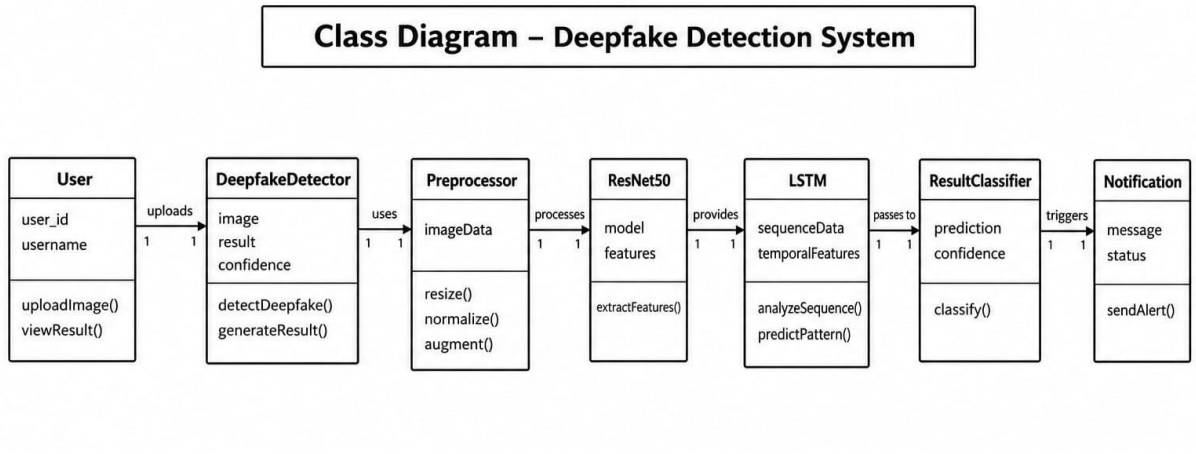



Figure 4.5

## 4.6 ACTIVITY DIAGRAM

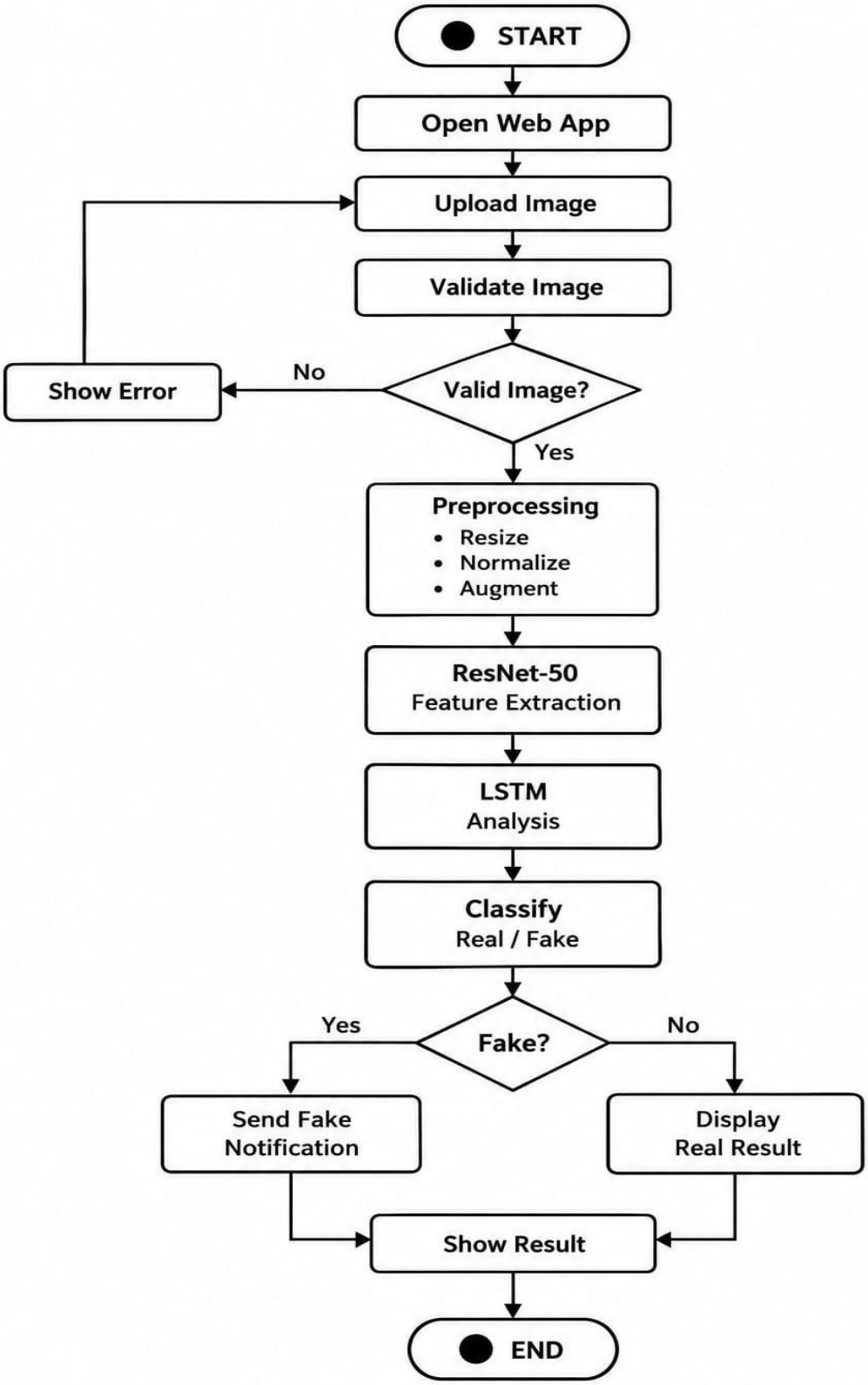



Figure 4.6

## 4.7 STATE FLOW DIAGRAM

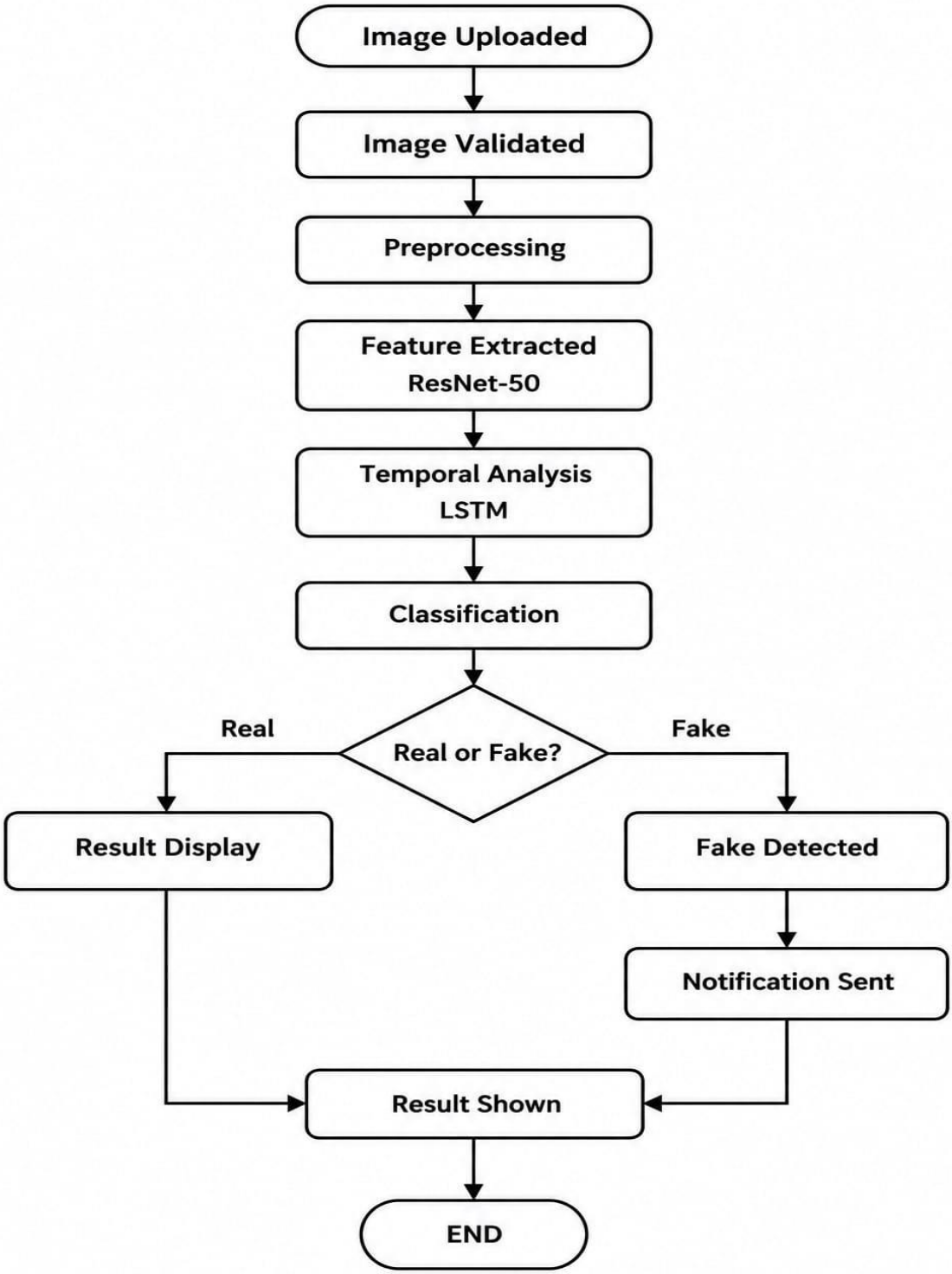



Figure 4.7

# IMPLEMENTATION

# CHAPTER 5

# IMPLEMENTATION

## 5.1 SYSTEM IMPLEMENTATION

### 5.1.1 Front-End Implementation

The front-end provides an interface for users to interact with the deepfake detection system. It is developed using HTML and CSS.

**Key points:**

- HTML is used to create the user interface.
- Provides an image upload option for users.
- Includes buttons for submitting the image.
- Displays the detection result to the user.
- CSS is used for layout, styling, spacing, and presentation**.**

### 5.1.2 Back-End Implementation

The back-end is developed using Python and Django. It manages the processing of uploaded images and connects the front-end with the deep-learning model and database.

**Key points:**

- Receives the image uploaded by the user.
- Performs image preprocessing.
- Sends the processed image to the detection model.
- Uses the ResNet-50 and LSTM hybrid model for analysis.
- Generates the Real/Fake prediction.
- Sends the result back to the front-end.

### 5.1.3 Database Implementation

The system uses MySQL for storing application-related information. MySQL client is used for communication between Django and MySQL.

**Key points:**

- Stores relevant application information.

- Stores detection-related information.
- Provides structured data management.
- Connects with the Django back-end.

### 5.1.4 Deep Learning Model Implementation

The deepfake detection process uses a hybrid ResNet-50 and LSTM architecture.

**Key points:**

- ResNet-50 extracts important spatial features from the image.
- LSTM analyses feature patterns and relationships.
- The hybrid architecture improves the detection process.
- The final output classifies the image as Real or Fake.

## 5.2 MODULES DESCRIPTION

| Module Name | Technique / Technology | Function |
|---|---|---|
| Data Collection Module | Dataset Collection | Collects real and deepfake images |
| Image Preprocessing Module | Resizing, Normalization, Augmentation | Prepares images for model processing |
| Feature Extraction Module | ResNet-50 / CNN | Extracts spatial features from images |
| Pattern Analysis Module | LSTM | Analyses feature patterns and sequences |
| Deepfake Detection Module | ResNet-50 + LSTM | Classifies images as Real or Fake |
| Database Module | MySQL + MySQLclient | Stores and manages system data |
| Result & Notification Module | Result Classification / Alert | Displays prediction and provides notification |
| Security Module | Encryption / Secure Handling | Protects uploaded and stored data |

Table 5.1: Overview of System Modules

## 5.2.1 MODULE 1: DATA COLLECTION

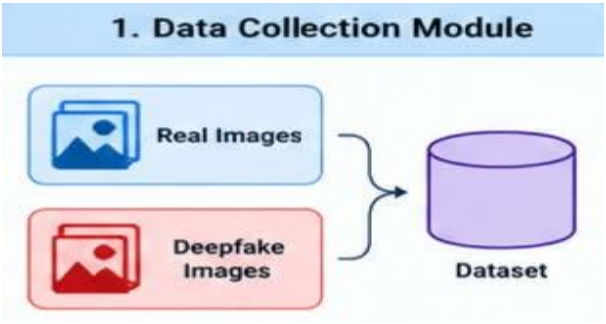



Figure 5.1: Module 1 – Data Collection

**Purpose:**

- To provide suitable data for training and testing the deepfake detection model.

**Features:**

- Collection of real images.
- Collection of manipulated/deepfake images.
- Organization of images for model processing.

## 5.2.2 MODULE 2: IMAGE PREPROCESSING

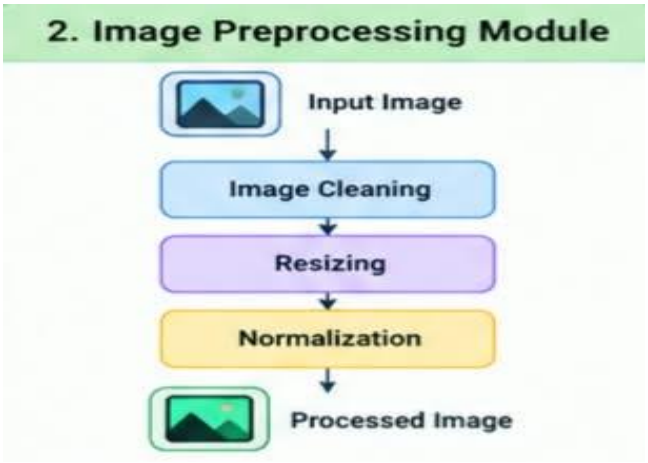



Figure 5.2: Module 2 – Image Preprocessing

**Purpose:**

- To convert images into a suitable format for deep-learning analysis.

**Features:**

- Image cleaning.
- Image resizing.
- Normalization.
- Data augmentation.

### 5.2.3 Module 3: Feature Extraction

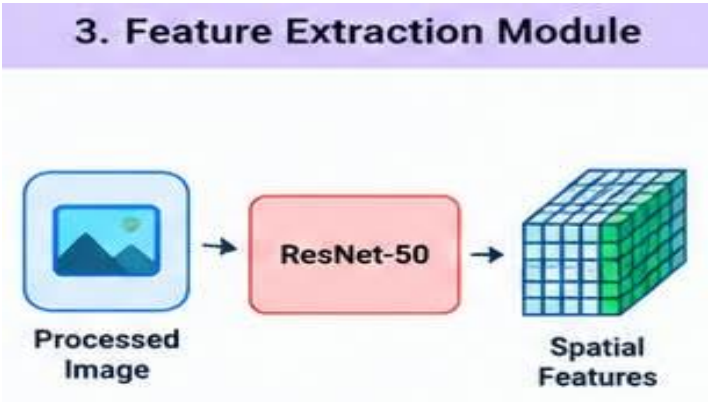



Figure 5.3: Module 3 – Feature Extraction

**Purpose:**

- To identify important visual features from the input image.

**Features:**

- Extracts high-level image features.
- Uses ResNet-50 architecture.
- Identifies spatial patterns.

### 5.2.4 Module 4: Pattern Analysis

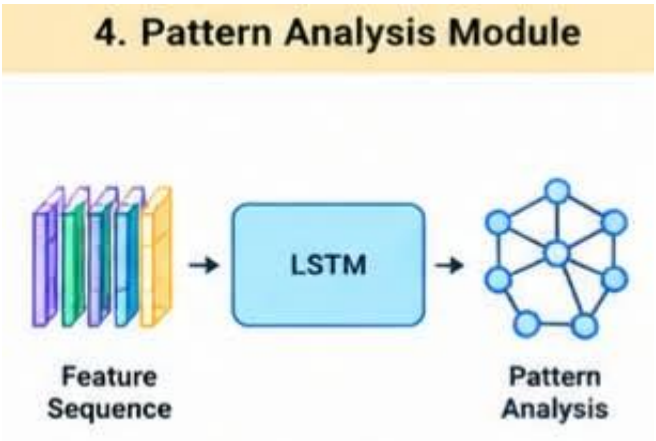



Figure 5.4: Module 4 – Pattern Analysis

**Purpose:**

- To analyse relationships and patterns in the extracted features.

**Features:**

- Processes feature sequences.
- Identifies important patterns.
- Supports deepfake classification.

### 5.2.5 Module 5: Deepfake Detection

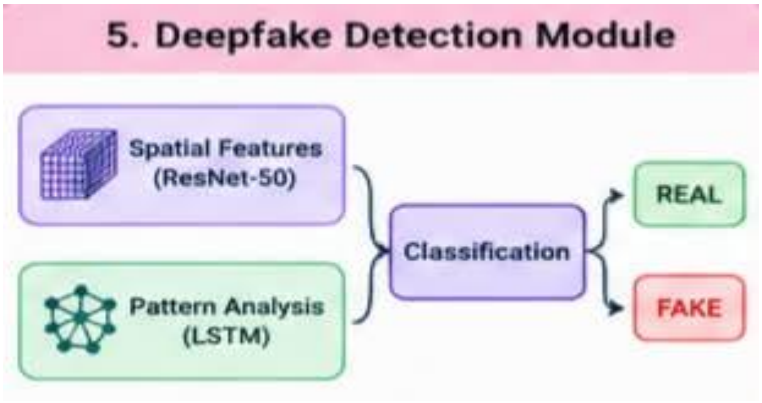



Figure 5.5: Module 5 - Deepfake Detection

**Purpose:**

- To determine whether the input image is authentic or manipulated.

**Features:**

- Combines ResNet-50 and LSTM.
- Analyses extracted features.
- Generates Real/Fake prediction.

### 5.2.6 Module 6: Database Management

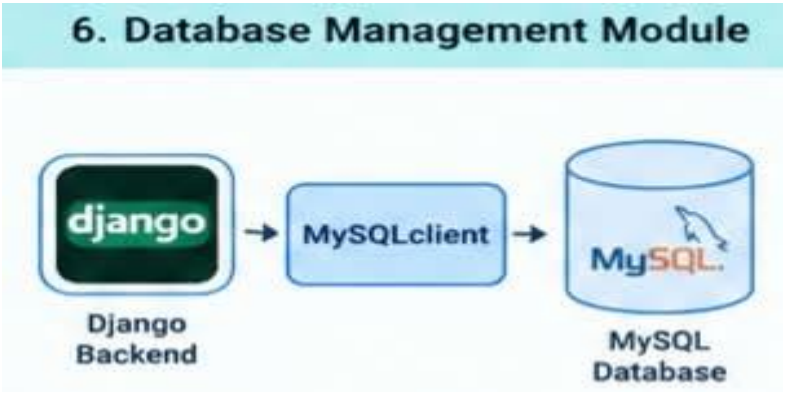



Figure 5.6: Module 6 – Database Management

**Purpose:**

- To store and manage relevant application and detection information.

**Features:**

- Data storage.
- Data retrieval.
- Database connectivity using MySQL client.

### 5.2.7 Module 7: Result & Notification




Figure 5.7: Module 7 – Result & Notification

**Purpose:**

- To communicate the detection result clearly to the user.

**Features:**

- Displays Real/Fake result.
- Provides detection information.
- Generates notification/alert for detected fake content.

### 5.2.8 Module 8: Security




Figure 5.8: Module 8 - Security

**Purpose:**

- To protect application data and uploaded content.

**Features:**

- Secure data handling.
- Protection of stored information.
- Encryption-related security measures.

# SYSTEM TESTING

# CHAPTER 6

# SYSTEM TESTING

## 6. SYSTEM TESTING

System testing is performed to verify that the proposed Deepfake Image Detection System works according to its requirements and produces the expected results. The testing process is used to identify errors, verify system functionality, and ensure that the integrated components work correctly. The testing covers functional behaviour, valid and invalid inputs, system responses, page navigation, and integration between different components.

## 6.1 Testing Strategy

The testing strategy focuses on checking the functionality of the system through manual and functional testing. The main objective is to verify that all required inputs are accepted correctly and that the system produces the expected output. The testing also checks whether the application pages and links work properly and whether the system responds without unnecessary delays.

The testing process includes checking valid inputs, invalid inputs, system functions, output results, and the interaction between integrated components. Black-box testing is used to verify the system behaviour without considering its internal implementation. Integration testing is also performed to ensure that different software components work together correctly.

## 6.2 Test Environment

The system is tested in the development environment using the software and technologies specified for the project. The application is developed using Python and Django, with MySQL used for database-related operations. The system can be accessed through a web browser, particularly Google Chrome. The testing environment provides the required platform for checking the image upload, processing, deepfake detection, result display, and other system functions.

The testing is performed by providing different inputs to the system and observing the generated responses. Both valid and invalid inputs are considered during functional testing to verify that the application behaves as expected.

## 6.3 Functional (Black-Box) Test Cases

Functional testing verifies whether the system functions according to the specified requirements. In black-box testing, the tester provides inputs to the system and checks the outputs without examining the internal implementation of the system.

| Test Case | Test Description | Expected Result |
|---|---|---|
| TC01 | Open the application | Application should open successfully |
| TC02 | Provide a valid image | Image should be accepted for processing |
| TC03 | Provide an invalid input | Invalid input should be rejected or handled properly |
| TC04 | Process an uploaded image | Image should be processed successfully |
| TC05 | Perform deepfake detection | System should generate a detection result |
| TC06 | Display the classification result | Real/Fake result should be displayed correctly |
| TC07 | Check notification for a fake image | Notification should be generated when a fake image is detected |
| TC08 | Check application links/pages | Required pages and links should open correctly |
| TC09 | Check system response | Screen, messages, and responses should not be unnecessarily delayed |
| TC10 | Check integrated components | Different components should interact without errors |

Table 6.1: Functional (Black-Box) Test Cases

The test cases are designed around the report's stated functional-testing objectives: valid and invalid input handling, exercising system functions, checking outputs, and verifying page/link behaviour.

## 6.4 Test Result Summary

The testing process was performed to verify the functionality and integration of the proposed system. The functional tests were used to check input handling, system operations, output generation, and application navigation. Integration testing was used to verify that the different software components interact correctly.

According to the testing results documented in the report, all the mentioned test cases passed successfully and no defects were encountered. Acceptance testing was also carried out to verify that the system satisfies the required functional requirements.

Therefore, the testing results indicate that the proposed system performs its intended functions and that the integrated components operate consistently within the tested environment.

# RESULTS & DISCUSSIONS

# CHAPTER 7

# RESULTS & DISCUSSIONS

## 7.1 FUNCTIONAL REVIEW

The system was reviewed to verify whether the major functions operate according to the project requirements.

**Observations:**

- Image upload functionality works as intended.
- Input images are processed through the preprocessing stage.
- ResNet-50 extracts important image features.
- LSTM analyses the extracted feature patterns.
- The system provides **Real/Fake** classification.
- Detection results are displayed to the user.
- Fake-image notification alerts are provided.
- The documented integration and acceptance tests were successfully completed without defects.

## 7.2 SECURITY ANALYSIS

Security measures are included to protect the data handled by the system.

**Observations:**

- Uploaded content is handled securely.
- Encryption techniques are considered for data transmission and storage.
- Sensitive information is protected from unauthorized access.
- Secure data handling supports the reliability of the detection system.

## 7.3 PERFORMANCE ANALYSIS

The performance of the system is based on its ability to process images and generate detection results efficiently.

**Observations:**

- Preprocessing standardizes the input images.

- ResNet-50 provides high-level feature extraction.
- LSTM analyses feature patterns.
- The hybrid architecture supports reliable deepfake detection.
- The system is designed with real-time processing capability.

## 7.4 USABILITY OBSERVATIONS

The system provides a simple interface for users to upload images and view the detection results.

**Observations:**

- Simple image upload process.
- Clear detection result.
- Easy-to-understand Real/Fake classification.
- Notification alerts provide immediate information.
- The system can be useful for media verification and digital forensics.

## 7.5 SAMPLE OUTPUTS

The system produces outputs based on the input image provided by the user.

**Sample Output 1 – Real Image:**

Input Image → Model Analysis → RESULT: REAL

**Sample Output 2 – Fake Image:**

Input Image → Model Analysis → RESULT: FAKE

↓

Notification Alert

**Sample Output 3 – Invalid Input:**

No Image Selected

↓

Please Upload an Image,

# LIMITATIONS AND RISK ANALYSIS

# CHAPTER 8

# LIMITATIONS AND RISK ANALYSIS

## 8.1 Password Strength Dependency

The security of the system depends partly on the strength of the password used by the user. Weak or easily predictable passwords may reduce the overall security of protected data. If a simple password is used, it may be easier for an unauthorized person to gain access. Therefore, users should use strong passwords containing a combination of letters, numbers, and special characters. Password strength remains an important limitation because the system cannot provide complete protection when users choose weak credentials.

## 8.2 Local Share Storage Risk

The system stores shared files or data locally, which may introduce security and availability risks. If the local storage location is accessed by an unauthorized person, the stored information may be exposed or modified. Data may also be affected if the local storage is accidentally deleted, corrupted, or becomes unavailable. Proper access control and secure storage practices are therefore important for protecting shared information.

## 8.3 No Built-in Key Store

The system does not provide a dedicated built-in key storage mechanism for securely managing encryption keys. If encryption keys are not stored and managed securely, protected data may become difficult to access or may be exposed to unauthorized users. This limitation places additional responsibility on the system or administrator to maintain and protect the required keys securely.

## 8.4 Denial of Service via Share Deletion

The system may face an availability risk if shared resources are deleted or removed repeatedly. Unauthorized or excessive deletion of shared resources can prevent legitimate users from accessing the required data. This can affect the availability of the system and interrupt normal operations. Appropriate access restrictions and controlled deletion mechanisms can help reduce this risk

# LITERATURE SURVEY

# CHAPTER 9

# LITERATURE SURVEY

## 9.1 Deepfake Image Detection Using Recurrent Neural Networks

**Author(s):** Afchar, D., Nozick, V., Yamagishi, J., & Echizen, I.

**Year:** 2018

This study presents a lightweight deepfake detection approach using a combination of Convolutional Neural Network (CNN) and Recurrent Neural Network (RNN) techniques. The method focuses on identifying temporal inconsistencies present in manipulated image frames. By analyzing these inconsistencies, the system can distinguish between authentic and manipulated content. The approach also aims to provide effective detection while maintaining relatively low computational requirements.

## 9.2 Exposing Deep Fakes Using Biological Signals

**Author(s):** Agarwal, S., Farid, H., Gu, Y., He, M., Nagano, K., & Li, H.

**Year:** 2019

This study proposes a deepfake detection method based on biological signals present in facial content. It analyzes subtle characteristics such as eye blinking and facial micro expressions. These natural facial signals may contain inconsistencies when the content is artificially generated or manipulated. Therefore, analyzing such biological characteristics can help identify deepfake content.

## 9.3 Deep Fake Detection with Spatiotemporal Features

**Author(s):** Sabir, E., Cheng, J., Jaiswal, A., & Abd-Almageed, W.

**Year:** 2019

This research focuses on extracting spatiotemporal features from image frames for detecting manipulated regions. The approach combines CNNs for extracting visual features with LSTM networks for analyzing patterns across the extracted information. The hybrid CNN-LSTM approach provides a method for identifying deepfake characteristics and improving the robustness of detection.

## 9.4 FaceForensics++: Learning to Detect Manipulated Facial Images

**Author(s):** Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner,M.

**Year:** 2019

This work introduces FaceForensics++, a comprehensive dataset and benchmark for detecting manipulated facial content. It provides different types of manipulated facial data that can be used for training and evaluating deepfake detection models. The study compares different detection approaches and highlights the importance of robust training for detecting various forms of facial manipulation.

## 9.5 DeepFake Detection: A Survey

**Author(s):** Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia

This survey provides an overview of different approaches used for deepfake detection. It helps in understanding the development of deepfake detection techniques and the challenges involved in identifying manipulated digital content. The study provides a foundation for comparing existing detection methods and understanding the need for more reliable and robust detection systems.

# APPENDICES

# CHAPTER 10

# APPENDIX A – SOURCE CODINGS

## 10.1 TECHNOLOGY STACK

```
import streamlit as st
import numpy as np
import pickle
import os
from PIL import Image, ImageOps
from streamlit_image_select import image_select
from tensorflow.keras.models import model_from_json
import time
st.set_page_config(layout="wide")
st.title("Deepfake Detector:")
# function to load and cache pretrained model
@st.cache_resource()
def load_model():
    path = "../dffnetv2B0"
    # Model reconstruction from JSON file
    with open(path + '.json', 'r') as f:
        model = model_from_json(f.read())
    model.load_weights(path + '.h5')
    return model
# function to preprocess an image and get a prediction from the model
def get_prediction(model, image):
    open_image = Image.open(image)
    resized_image = open_image.resize((256, 256))
    np_image = np.array(resized_image)
```

```
    reshaped = np.expand_dims(np_image, axis=0)
    predicted_prob = model.predict(reshaped)[0][0]
    if predicted_prob >= 0.5:
        return f"Real, Confidence: {str(predicted_prob)[:4]}"
    else:
        return f"Fake, Confidence: {str(1 - predicted_prob)[:4]}"
# generate selection of sample images
@st.cache_data()
def load_images():
  real_images = ["images/Real/" + x for x in os.listdir("images/Real/")]
  fake_images = ["images/Fake/" + x for x in os.listdir("images/Fake/")]
  image_library = real_images + fake_images
  image_selection = np.random.choice(image_library, 20, replace=False)
  return image_selection
# load model
classifier = load_model()
images = load_images()
def game_mode():
  st.header("Game Mode")
  st.subheader("Can you beat the model?")
  # from streamlit_image_select docs - https://github.com/jrieke/streamlit-image-select
  selected_image = image_select(
    "Click on an image below to guess if it is real of fake:",
    images,
    return_value="index")
  prediction = get_prediction(classifier, images[selected_image])
  true_label = 'Fake' if 'fake' in images[selected_image].lower() else 'Real'
  #st.text(true_label)
  st.subheader("Is this image real or fake?")
```

```
  st.image(images[selected_image])
 if st.button("It's Real"):
    st.text("You guessed:")
    st.subheader("*Real*")
    st.text("The Deepfake Detector model guessed...")
    time.sleep(1)
    st.subheader(f"*{prediction}*")
    st.text("The truth is...")
    time.sleep(1)
    st.subheader(f"***It's {true_label}!***")
  if st.button("It's Fake"):
    st.text("You guessed:")
    st.subheader("*Fake*")
    st.text("The Deepfake Detector model guessed...")
    time.sleep(1)
    st.subheader(f"*{prediction}*")
    st.text("The truth is...")
    time.sleep(1)
    st.subheader(f"***It's {true_label}!***")
def detector_mode():
  st.header("Detector Mode")
  st.subheader("Upload an Image to Make a Prediction")
  # upload an image
  uploaded_image = st.file_uploader("Upload your own image to test the model:", type=['jpg',
'jpeg'])
  # when an image is uploaded, display image and run inference
  if uploaded_image is not None:
    st.image(uploaded_image)
    st.text(get_prediction(classifier, uploaded_image))
```

```
page = st.sidebar.selectbox('Select Mode',['Detector Mode','Game Mode'])
if page == 'Game Mode':
  game_mode()
else:
  detector_mode()
```

# APPENDIX B – SAMPLE SCREENSHOTS

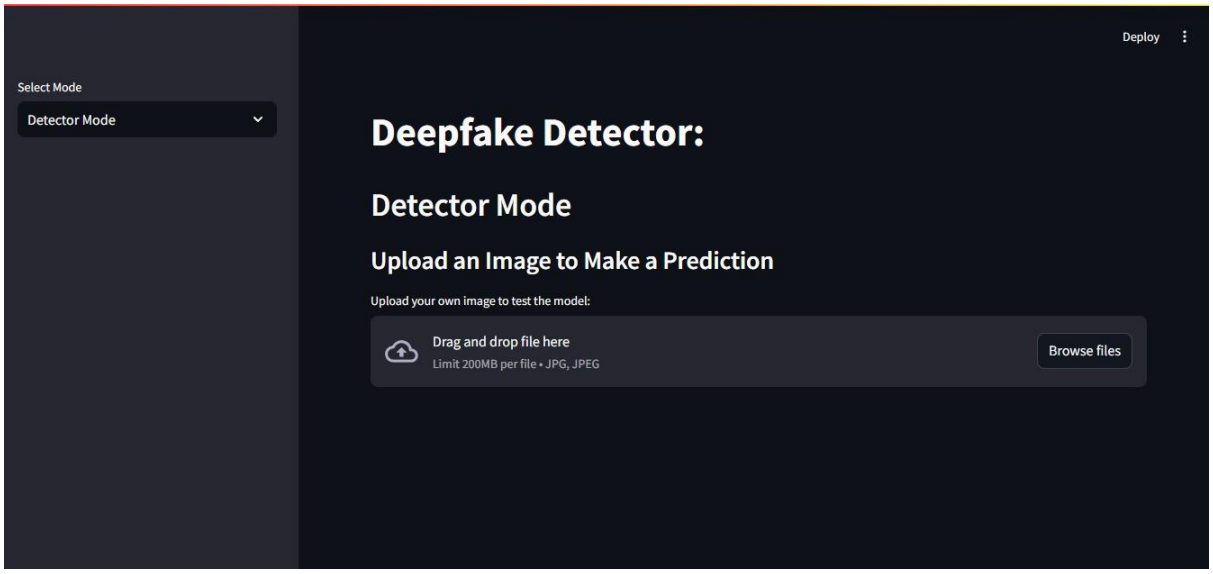



Figure 10.1

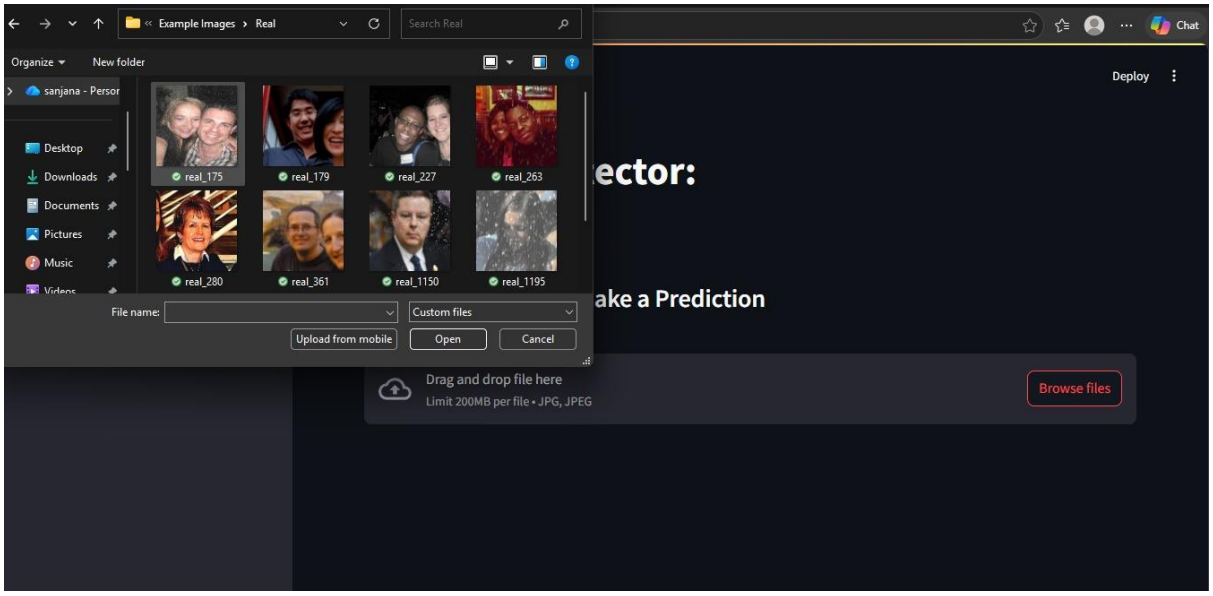



Figure 10.2

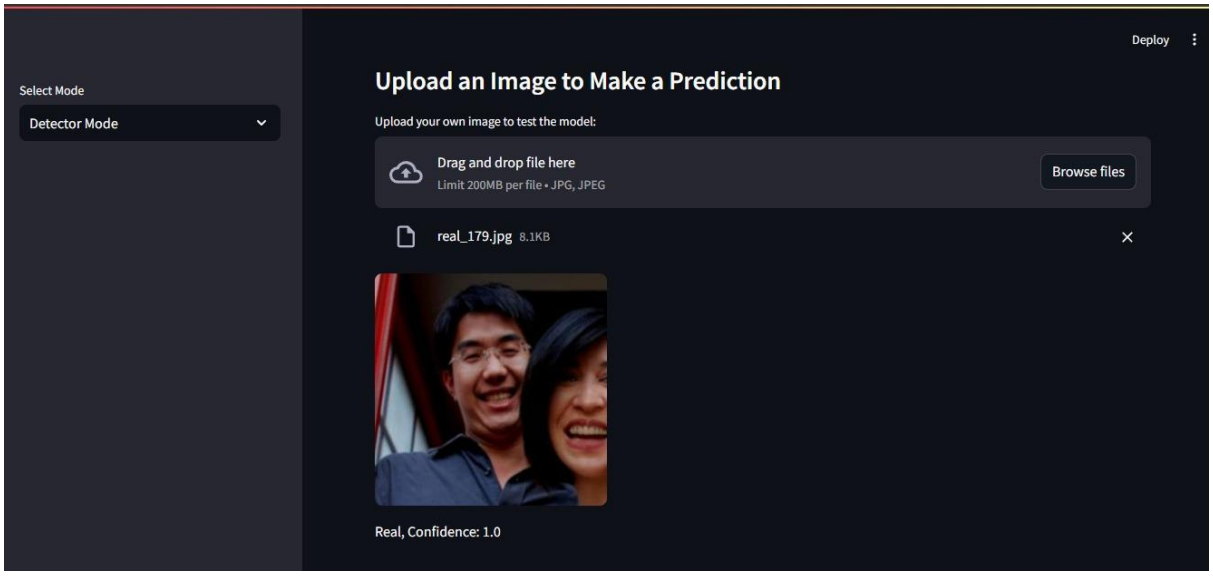



Figure 10.3

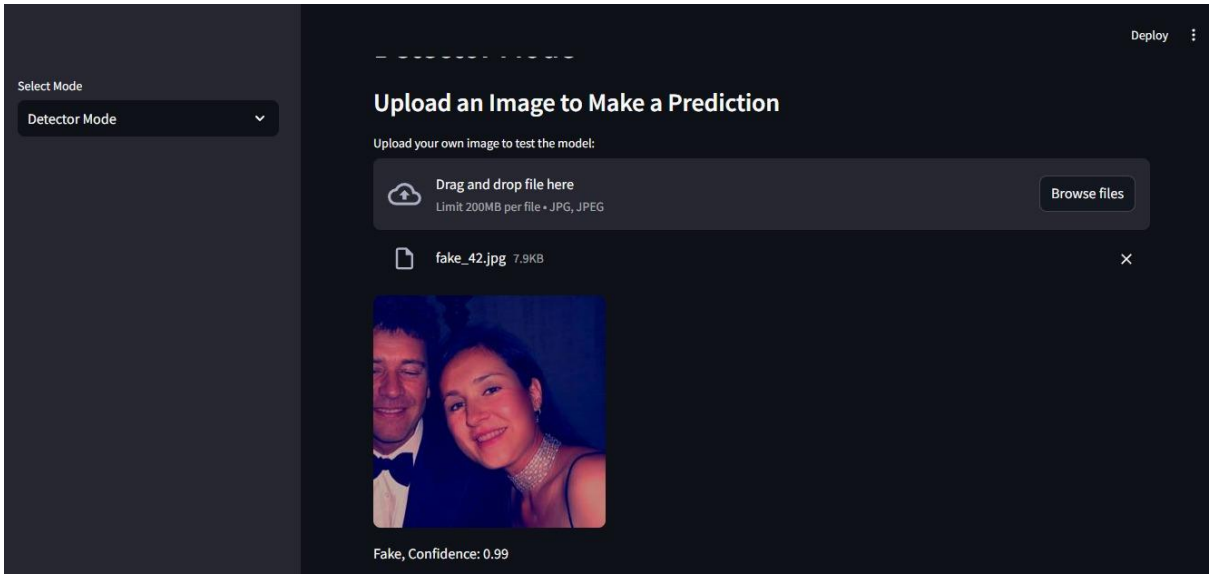



Figure 10.4

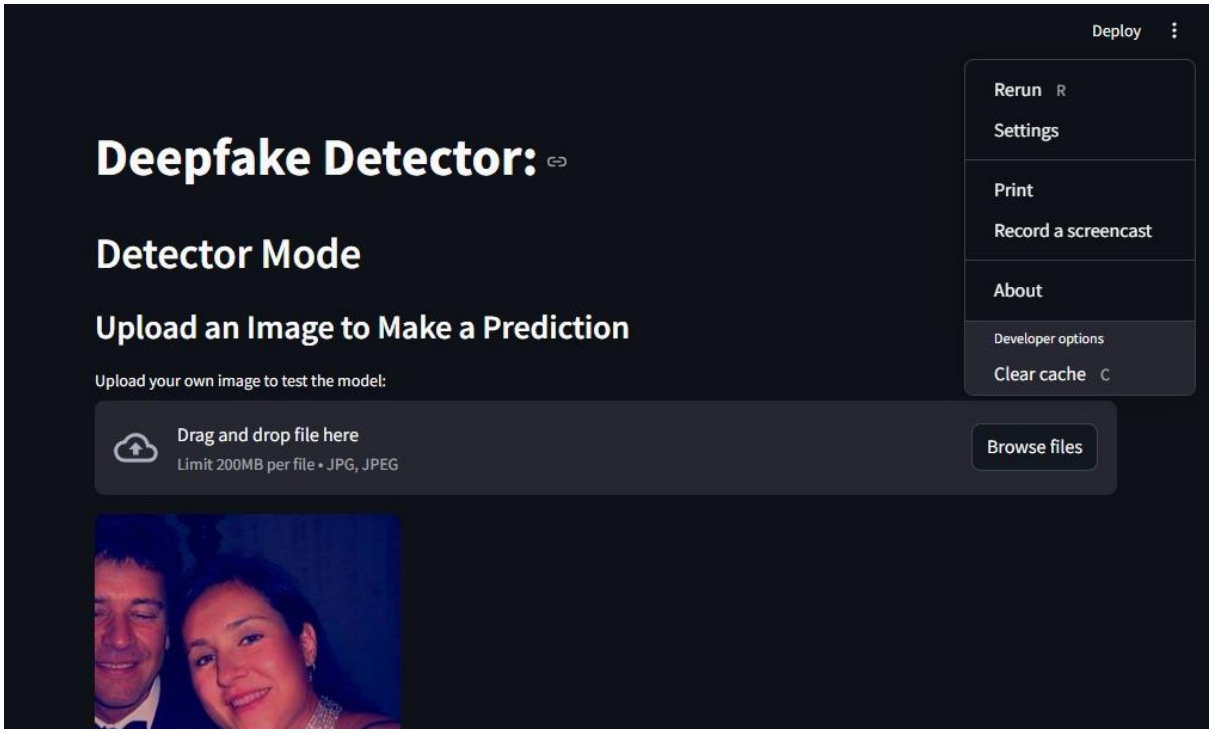

Deploy
Rerun R
Settings
Print
Record a screencast
About
Developer options
Clear cache C
Deepfake Detector:
Detector Mode
Upload an Image to Make a Prediction
Upload your own image to test the model:
Drag and drop file here
Limit 200MB per file • JPG, JPEG
Browse files

Figure 10.5

## 10.2 ERROR HANDLING STRATEGIES

Error handling is used to ensure that the deepfake detection system works reliably when unexpected or invalid inputs occur.

### 10.2.1 Invalid Image Handling

- Checks whether the uploaded file is a valid image.
- Displays an error message for unsupported files.
- Prevents invalid files from being processed.

### 10.2.2 Empty Input Handling

- Checks whether an image has been selected.
- Displays a message when no image is uploaded.
- Prevents unnecessary backend processing.

### 10.2.3 Image Processing Errors

- Handles errors during image reading, resizing, and normalization.
- Prevents corrupted images from affecting the detection process.
- Displays an appropriate processing error message.

### 10.2.4 Model Prediction Errors

- Handles failures during ResNet-50 and LSTM processing.
- Prevents incomplete predictions from being displayed.
- Provides an error message when model processing fails.

### 10.2.5 Database Errors

- Handles database connection and data-storage failures.
- Prevents system crashes when the database is unavailable.
- Displays an appropriate database error message.

### 10.2.6 Server Errors

- Handles unexpected Django backend errors.
- Maintains proper communication between frontend and backend.
- Provides user-friendly error messages instead of technical errors.

## APPENDIX C – GLOSSARY OF TERMS

| Term | Meaning |
|---|---|
| **Deepfake** | Artificially manipulated digital content created using AI techniques. |
| **Deep Learning** | A machine-learning approach based on multi-layer neural networks. |
| **CNN** | Convolutional Neural Network used mainly for processing image features. |
| **ResNet-50** | A deep neural network architecture used for high-level feature extraction. |
| **LSTM** | Long Short-Term Memory network used for analysing sequential patterns. |
| **Hybrid Model** | A model that combines ResNet-50 and LSTM for deepfake detection. |
| **Feature Extraction** | Process of identifying important characteristics from an image. |
| **Preprocessing** | Preparing input data through operations such as resizing and normalization. |
| **Classification** | Assigning an input image to a category such as Real or Fake. |
| **Dataset** | A collection of images used for training and testing the model. |

| | |
|---|---|
| **Django** | Python-based web framework used for backend development. |
| **MySQL** | Relational database management system used to store application data. |
| **MySQL client** | Database connector used to communicate between Python/Django and MySQL. |
| **HTML** | Markup language used to create the structure of the web interface. |
| **CSS** | Styling language used to design the web interface. |
| **Normalization** | Process of scaling image data into a suitable range for model processing. |
| **Data Augmentation** | Technique used to create variations of training images to improve model robustness. |
| **Encryption** | Technique used to protect data from unauthorized access. |
| **Prediction** | Output generated by the deep-learning model after analysing an input image. |
| **Notification Alert** | Message provided to the user when manipulated content is detected. |

Table 10.1: Glossary

# CONCLUSION AND FUTURE ENHANCEMENT

# CHAPTER 11
# CONCLUSION AND FUTURE ENHANCEMENT

## 11.1 CONCLUSION

The project "A Robust Framework for Deepfake Detection Using Advanced Neural Architectures and Generalization Techniques" presents a deep-learning-based approach for detecting manipulated images. The system combines ResNet-50 for spatial feature extraction and LSTM for pattern analysis to classify content as real or fake. Image preprocessing, secure data handling, database integration, and notification alerts are also included. The documented testing results show that the specified test cases were completed successfully without defects.

The project demonstrates how deep-learning techniques can be applied to support digital content verification, media monitoring, and digital forensics.

## 11.2 LEARNING OUTCOMES

Through this project, the following skills and knowledge were gained:

- Understanding of deepfake detection concepts.
- Knowledge of CNN and deep-learning architectures.
- Understanding and implementation of ResNet-50 for feature extraction.
- Understanding of LSTM for pattern analysis.
- Knowledge of image preprocessing techniques.
- Experience with Python and Django for application development.
- Understanding of MySQL database integration.
- Knowledge of frontend development using HTML and CSS.
- Understanding of software testing and error handling.
- Knowledge of security and notification mechanisms.

## 11.3 FUTURE ENHANCEMENT

The system can be further improved to make it more flexible and effective.

- Extend detection from still images to video content.
- Support a wider range of deepfake generation techniques.
- Improve generalization using larger and more diverse datasets.
- Explore advanced deep-learning architectures for better detection.
- Improve real-time detection performance.
- Enhance the notification and alert system.
- Develop a mobile or cloud-based version of the application.

# REFERENCES

# CHAPTER 13

# REFERENCES


[1]. E. Kim and S. Cho, "Exposing Fake Faces through Deep Neural Networks Combining Content and Trace Feature Extractors," in IEEE Access, vol. 9, pp. 123493-123503, 2021, doi: 10.1109/ACCESS.2021.3110859.

[2]. T. Jung, S. Kim and K. Kim, "Deep Vision: Deepfakes Detection Using Human Eye Blinking Pattern," in IEEE Access,vol.8,pp.8314483154,2020,doi:10.11 09/ACCESS.20 20.2988660.

[3]. A. H. Khalifa, N. A. Zaher, A. S. Abdallah and M. W. Fakhr, "Convolutional Neural Network Based on Diverse Gabor Filters for Deepfake Recognition," in IEEE Access, vol.10,pp.2267822686,2022,doi:10.1109/AC CESS.2022.31 52029.

[4]. Rana, Md Nobi, Mohammad, Murali, Beddhu, Sung, Andrew, 2022/01/01 Deepfake Detection: A Systematic LiteratureReviewVL10DO10.1109/ACCESS .2022.3154404

[5]. H. R. Hasan and K. Salah, "Combating Deepfake Images Using Blockchain and Smart Contracts," in IEEE Access, vol.7,pp.4159641606,2019,doi:10.1109/ACC ESS.2019.290 5689.

[6]. N. Waqas, S. I. Safie, K. A. Kadir, S. Khan and M. H. Kaka Khel, "Deepfake Image Synthesis for Data Augmentation," in IEEE Access, vol. 10, pp. 80847-80857, 2022, doi: 10.1109/ACCESS.2022.3193668.

[7]. L. Guarnera, O. Giudice and S. Battiato, "Fighting Deepfake by Exposing the Convolutional Traces on Images," in IEEE Access, vol. 8, pp. 165085-165098, 2020, doi: 10.1109/ACCESS.2020.3023037.

[8]. Lin Zhang, Xin Wang, Erica Cooper, Nicholas Evans, and Junichi Yamagishi. 2022. The Partial Spoof Database and Countermeasures for the Detection of Short Fake Speech Segments Embedded in an Utterance. IEEE/ACM Trans. Audio, Speech and Lang. Proc. 31 (2023), 813–825. https://doi.org/10.1109/TASLP.2022.323323 6

[9]. -J. Yu, S. -H. Nam, W. Ahn, M. -J. Kwon and H. -K. Lee, "Manipulation Classification for JPEG Images Using Multi-Domain Features," in IEEE Access, vol. 8, pp. 210837210854,2020,doi:

10.1109/ACCESS.2020.3037735. [10]. A. Mary and A. Edison, "Deep fake Detection using deep learning techniques: A Literature Review," 2023 International Conference on Control, Communication and Computing (ICCC), Thiruvananthapuram, India, 2023, pp. 1-6, doi: 10.1109/ICCC57789.2023.10164881.

[11]. H. Zhao, et al., "Multi-attentional Deepfake Detection," in 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Nashville, TN, USA,2021pp.21852194.doi:10.1109/CVPR4 6437.2021.002 22

[12]. H. Mamtora, K. Doshi, S. Gokhale, S. Dholay and C. Gajbhiye, "Image Manipulation Detection and Localization Using Deep Learning," 2020 2nd International Conference on Advances in Computing, Communication Control and Networking (ICACCCN), Greater Noida, India,2020,pp.241248,doi:10.1109/ICACCC N51052.2020.9 362923.

[13]. C. Li, et al., "A Continual Deep fake Detection Benchmark: Dataset, Methods, and Essentials," in 2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), Waikoloa, HI, USA, 2023 pp. 1339-1349.doi: 10.1109/WACV56688.2023.00139

[14]. Korshunova, W. Shi, J. Dambre and L. Theis, "Fast Face-Swap Using Convolutional Neural Networks," in 2017 IEEE International Conference on Computer Vision (ICCV), Venice,Italy,2017pp.36973705.doi:10.1109/I CCV. 2017.397

[15]. R. Raghavendra, K. B. Raja, S. Venkatesh and C. Busch, "Transferable Deep-CNN Features for Detecting Digital and PrintScanned Morphed Face Images," 2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Honolulu, HI, USA, 2017, pp. 1822-1830, doi: 10.1109/CVPRW.2017.228.